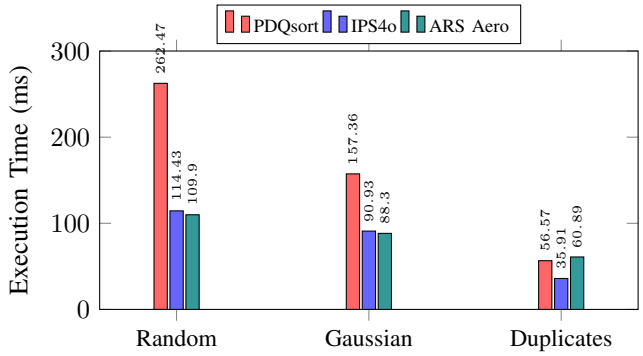
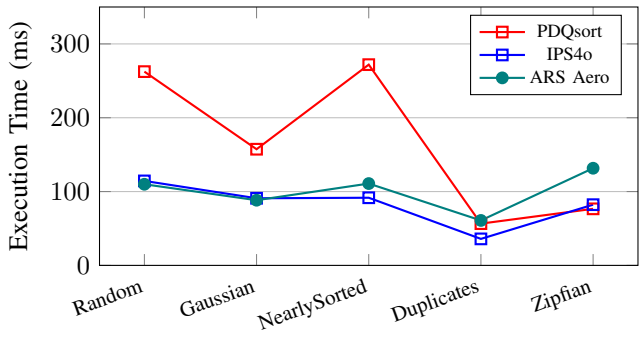
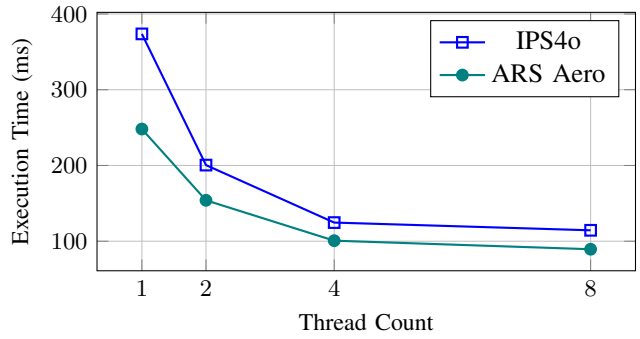


(a) Distribution performance ($N = 10^7$)

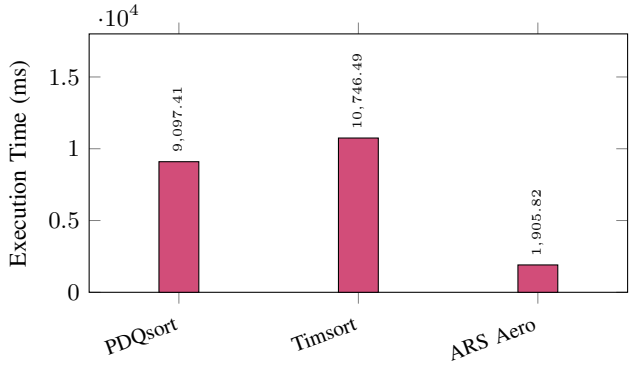


(b) Entropy response ($N = 10^7$)

Fig. 4: Distribution and Entropy Performance Evaluation. ARS Aero maintains competitive performance across uniform and skewed distributions, while standard comparison algorithms exploit low-entropy duplicate patterns.



(a) Parallel Scalability ($N = 10^7$)



(b) Sorting 10,000,000 Random Strings

Fig. 5: Scaling Performance under Parallel Cores and Type Complexity. ARS Aero maintains scaling advantages across multi-threaded integer and complex-type string workloads.

However, the framework remained competitive while preserving stable throughput characteristics across large-scale workloads.

F. Robustness Analysis

To evaluate the robustness for ARS framework, we examine the *Effective Constant Factor*, defined as the execution time per element (T/N). As shown in fig. 6a, standard $O(N \log N)$ sorters like PDQsort exhibit a logarithmic growth in per-element cost as N increases.

In contrast, ARS Aero shows a "Sampling Penalty" at small scales ($N = 10^4$), where the fixed cost of quantile estimation and table construction is more pronounced. However, the factor stabilizes at ≈ 7.7 ns per element for $N \geq 10^5$. This plateau indicates that the computational work per element becomes relatively independent of the total dataset size for large-scale workloads.

Another observed point at fig. 6a is the massive spike in cost at 10^4 elements and this represents the high "Fixed Costs" (initializing the Rayon thread pool, building the 1024-entry Quantile Table, and generating the Histogram) that ARS have, however once N is large enough the fixed setup costs become negligible. The execution time is now dominated by the single-pass "Scatter Phase".

Beyond median execution time, the predictability of a sorting algorithm is critical for real-time systems. Table I provides the Median execution time and Coefficient of Variation (CoV) at $N = 10^7$. ARS Aero maintains a CoV $< 3\%$ for most distributions, indicating that its performance is primarily a function of memory bandwidth rather than algorithmic branch sensitivity.

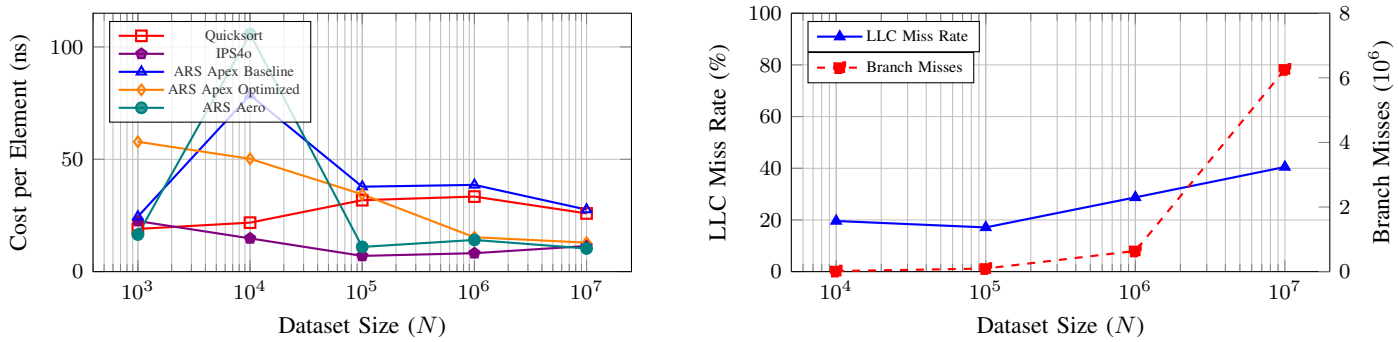
TABLE I: Latency Stability ($N = 10^7$, Median Time ms)

Distribution	PDQsort	ARS Aero	CoV (Aero)
Random	262.47	109.90	2.9%
Gaussian	157.36	88.30	3.4%
Duplicates	56.57	60.89	2.4%
NearlySorted	271.92	110.85	3.0%

G. Microarchitectural Analysis

Beyond algorithmic complexity, the behavior of ARS is mainly shaped by memory hierarchy interactions and branch efficiency.

The scatter pipeline minimizes unpredictable branching by replacing comparison-driven partition traversal with fixed-cost spatial classification. This substantially reduces the frequency of branch-dependent execution stalls relative to recursive comparison sorters.



(a) Effective Constant Factor (T/N)

(b) Hardware Utilization Trends

Fig. 6: Algorithmic Complexity and Hardware Utilization Scaling. As dataset size N increases, ARS constant factor stabilizes due to cache-conscious layout optimization despite scaling LLC miss rates.

Furthermore, the use of software write-combining buffers converts highly fragmented random writes into contiguous cache-line bursts. This improves Store Buffer utilization, reduces Translation Lookaside Buffer (TLB) pressure, and increases effective memory-bandwidth saturation during the scatter phase we can clearly observe this at fig. 7.

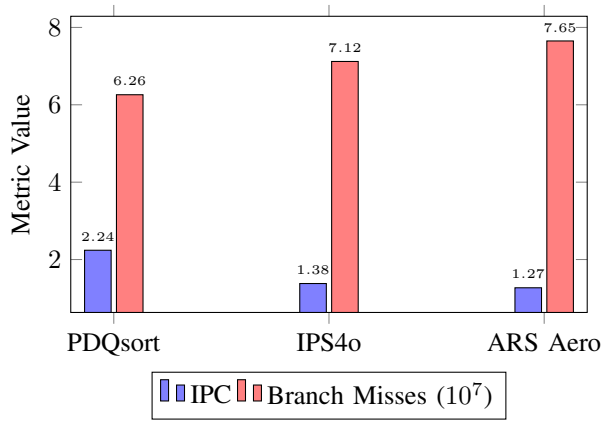


Fig. 7: Hardware Efficiency Comparison. ARS Aero achieves faster completion times despite a lower IPC due to a massive reduction in total instructions executed (bypassing comparison-tree branch overhead).

While standard PDQsort achieves a higher IPC (2.24) due to its extremely tight comparison loops, it evaluates 20×10^6 comparisons for a million elements, each serving as a potential branch stall. In contrast, ARS Aero operates at a lower IPC (≈ 1.27) but completes the sort in half the time. This indicates that ARS executes fewer total branches, but a slightly higher absolute number of mispredictions due to scatter logic.

Additionally, ARS Aero hits the "Memory Wall" earlier than comparison sorts. As seen in table II, the framework reaches a peak throughput of $\approx$ 1477 MB/s for 64-bit integers. On the Zen 3 architecture, performance is optimized for a single NUMA node where all 8 cores share a 16MB L3 cache. In multi-socket or multi-NUMA environments, crossing NUMA boundaries constitutes the primary performance bottleneck for spatial scatter-partitioning due to remote memory access latency, which necessitates strict NUMA-aware thread affinity. Our results show that ARS's performance remains stable as long as the dataset fits within the aggregate L3 cache of the CCD; once the dataset exceeds this ($N > 10^5$), the LLC miss rate increases from 13% to 59%, and the algorithm's throughput becomes strictly limited by DRAM bandwidth.

Another notable behavior is observed at a dataset size of $N = 10^4$, which lies in the "Thread-Pool Uncanny Valley." Here, the workload is large enough to activate Rayon's parallel task-splitting logic, but the actual sorting work (10,000 items) is so small that thread synchronization overhead dominates the execution time.

TABLE II: Throughput Scaling and Cache Pressure (Random i64)

N	Throughput (MB/s)	LLC Miss Rate	IPC
10^3	926.1	77.4%	2.12
10^4	144.2	17.7%	0.57
10^5	1381.5	13.4%	0.99
10^6	1075.7	47.7%	1.39
10^7	1476.6	59.0%	1.27

Additionally to confirm our architectural thesis, we should observe the fig. 6b, in which we can observe that LLC Miss Rate surge from 19.59% to 40.49% as N scales from 10^4 to 10^7. This confirms that ARS becomes memory-bound once the dataset exceeds the 16MB L3 cache.

fig. 6b confirm that ARS trades cache locality for branch stability. It shows that while the CPU has to wait for memory (high LLC misses), it never has to restart the pipeline (low branch misses).

H. Streaming Evaluation

The Exp D architecture was evaluated under continuous streaming ingestion workloads to measure sustained throughput, latency stability, and bounded memory growth.

Incoming elements were accumulated through epoch-based micro-batching before being dispatched to concurrent background consolidation workers. This decoupled ingestion throughput from merge latency while preserving sequential memory-access behavior.

TABLE III: Exp D Streaming Performance Audit ($N = 10^8$)

Workload Pattern	Throughput	P50 (μs)	P99 (μs)
Uniform	24.29M ops/s	363	22,575
Zipfian	9.31M ops/s	1	133,759
Bursty	203.99M ops/s	¡1	424

Experimental observations at an extreme scale of 100 million elements indicate that Exp D maintained stable ingestion throughput under burst-heavy workloads while preventing unbounded run proliferation through proactive spatial consolidation.

Unlike traditional external merge pipelines that defer consolidation until late execution stages, Exp D continuously merged spatially adjacent runs in the background. This reduced the final aggregation overhead and stabilized latency under sustained streaming pressure.

Pareto trade-off analysis observed at table III and fig. 8indicates that a micro-batch threshold of $N_e \approx 32,768$ provides the optimal balance, maximizing throughput while preventing unmanageable P99 latency spikes during background merges.

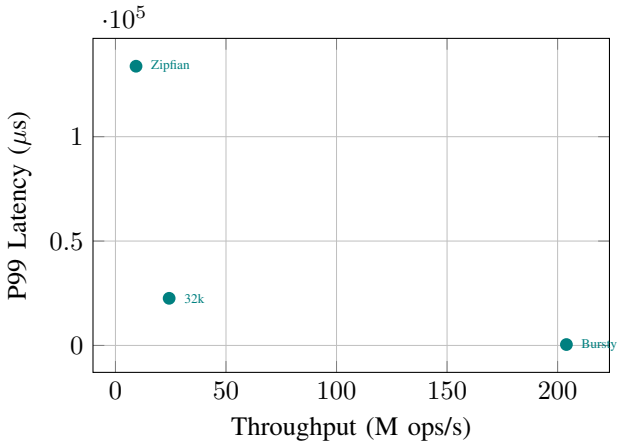


Fig. 8: Streaming Pareto Frontier ($N = 10^8$). Performance varies based on data pattern; bursty workloads allow the shadow pipeline to achieve maximum efficiency.

The bounded work-queue design also prevented uncontrolled memory growth under ingestion saturation by introducing controlled back-pressure once consolidation capacity was exceeded.

I. Limitations and Trade-offs

Despite its throughput advantages, ARS introduces several architectural trade-offs.

First, the framework increases memory traffic relative to cache-efficient comparison sorters due to the additional analysis and scatter passes. Although this overhead is partially amortized through sequential access behavior and write-combining efficiency, memory-bandwidth limitations may become dominant on bandwidth-constrained systems.

Second, ARS currently relies on auxiliary scatter buffers, resulting in an $O(n)$ additional memory footprint. While this design significantly improves throughput by enabling contiguous writes and predictable memory access patterns, it increases total DRAM consumption relative to in-place sorting algorithms.

Third, workloads dominated by duplicates or highly clustered key ranges may still produce localized refinement imbalance despite Aero's quantile-adaptive classification. Under these conditions, refinement cost may increasingly resemble comparison-based sorting behavior.

Finally, throughput scaling remains partially dependent on cache capacity and memory hierarchy behavior. As refinement partitions exceed cache residency thresholds, DRAM latency becomes increasingly visible, motivating the recursive refinement strategies explored in Exp A, however each level of recursive mapping incurs a fresh "Sampling Penalty." Constructing the reciprocal multiplier and histogram for every nested bin introduces significant constant-factor overhead.

J. Comparative Discussion

Overall, the experimental results suggest that ARS is best interpreted not as a universal replacement for comparison-based sorting, but as a hardware-conscious spatial classification framework optimized for large-scale parallel and streaming workloads.

Compared to branch-optimized comparison sorters such as PDQsort, ARS reduces dependence on unpredictable branching and achieves improved throughput under high-entropy workloads. Compared to sample-based parallel sorters such as IPS4o, Aero provides more stable occupancy behavior under skewed distributions through constant-time quantile classification.

The strongest performance characteristics of the framework emerge under conditions where:

- branch misprediction penalties dominate execution cost,
- memory-level parallelism becomes critical,
- workloads exhibit significant entropy or skew,
- or continuous streaming ingestion is required.

These results suggest that hardware-conscious spatial classification represents a viable alternative to comparison-driven sorting for modern high-throughput multi-core data processing systems.

VI. Limitations

Despite the favorable throughput characteristics demonstrated throughout this study, ARS remains subject to several architectural limitations and trade-offs that influence its applicability across different workload classes and hardware environments.

A. Memory Overhead

The current Aero implementation relies on auxiliary scatter buffers, spatial key caches, histogram structures, and temporary refinement storage. As a result, the framework requires an additional $O(n)$ memory footprint beyond the input dataset. While this design enables contiguous memory writes, contention-free partitioning, and improved cache utilization, it

increases total DRAM consumption relative to in-place comparison sorters such as Quicksort and PDQsort. Consequently, ARS may be less suitable for memory-constrained environments where auxiliary allocation costs outweigh throughput gains.

B. Duplicate-Dominated Workloads

Spatial classification performs best when the projected key space exhibits sufficient dispersion. Under highly duplicate-dominated or heavily clustered distributions, a large fraction of elements may collapse into a small number of spatial partitions. This increases local refinement costs and reduces the effectiveness of occupancy balancing. Although Aero's quantile-adaptive mapping improves robustness under skewed distributions, duplicate-aware comparison sorters may still outperform the current ARS implementation in extreme duplicate-heavy scenarios.

C. Cache-Capacity Dependence

The efficiency of ARS is strongly influenced by cache residency. The framework assumes that refinement partitions remain sufficiently small to benefit from the processor cache hierarchy. As dataset scale increases, some partitions may exceed available cache capacity, exposing additional DRAM latency and reducing throughput efficiency. This limitation motivated the development of Exp A, which explored recursive cache-aware refinement strategies designed to preserve cache locality for very large datasets. However, such mechanisms are not currently incorporated into the primary Aero implementation.

D. Quantile Estimation Accuracy

Aero relies on a sampled approximation of the input distribution to construct its quantile lookup table. Although this approach substantially improves occupancy stability for non-uniform workloads, its effectiveness depends on the representativeness of the sampled data. Rapidly changing distributions, adversarial inputs, or highly localized data characteristics may reduce the accuracy of the approximation and lead to less balanced partition occupancy. Future adaptive sampling strategies may further improve robustness under such conditions.

E. Scope of Evaluation

The experimental evaluation presented in this work focuses on shared-memory multi-core systems. Distributed execution, NUMA-aware partitioning strategies, heterogeneous accelerators such as GPUs, and large-scale cluster deployments remain outside the scope of the current study. Consequently, the results should be interpreted primarily as evidence of ARS behavior within modern shared-memory environments rather than a universal characterization across all computing platforms.

F. Streaming Scope

The Exp D architecture demonstrates that spatial classification can be extended to support high-throughput streaming ingestion through epoch-based micro-batching and concurrent spatial consolidation. However, the current design targets append-oriented workloads and does not address persistence, fault tolerance, distributed stream coordination, or recovery semantics. These concerns are essential for production-grade stream processing systems and represent important directions for future investigation.

Overall, the limitations identified in this section do not invalidate the central contributions of ARS, but rather highlight the architectural trade-offs associated with transforming sorting into a hardware-conscious spatial classification problem. The framework exchanges additional memory usage and distribution sensitivity for improved throughput, reduced branch dependence, and scalable parallel execution, reflecting a deliberate design choice tailored toward modern multi-core data processing workloads.

VII. Future Directions

The ARS framework establishes a hardware-conscious spatial classification architecture for parallel and streaming data processing. While the present work focuses primarily on the Aero and Exp D implementations, several promising research directions remain open.

A. Recursive Cache-Aware Refinement

The experiments presented in this paper indicate that cache residency plays a critical role in determining refinement efficiency. As dataset sizes continue to grow, partition sizes may eventually exceed cache capacity, exposing additional memory latency and reducing throughput.

Exp A explored recursive spatial refinement as a mechanism for preserving cache-local execution by recursively repartitioning oversized bins. Future work may investigate adaptive cache-aware recursion strategies, dynamic refinement thresholds, and formal cache-complexity models capable of predicting optimal partition hierarchies across diverse hardware platforms.

B. Hierarchical Memory Staging

The scatter phase remains fundamentally constrained by memory-system behavior. Exp B demonstrated that software write-combining buffers can significantly improve memory-bus utilization by transforming fragmented writes into contiguous cache-line bursts.

Future research may explore multi-level staging architectures that explicitly leverage L1, L2, and LLC hierarchies, as well as NUMA-aware staging strategies designed for high-core-count processors and multi-socket systems.

C. Entropy-Adaptive Execution

One of the most promising exploratory directions is the development of workload-adaptive execution policies. Exp C demonstrated that runtime entropy estimation can be used to

dynamically select different execution strategies depending on dataset characteristics.

Future systems may extend this concept by incorporating online learning, adaptive cost models, or lightweight predictive heuristics capable of selecting partitioning factors, refinement strategies, and mapping functions automatically at runtime. Such mechanisms would transform ARS from a static architecture into a self-optimizing data-processing framework.

D. Distributed Spatial Classification

The current implementation targets shared-memory multi-core systems. However, the spatial partitioning model naturally exposes coarse-grained data parallelism that may be suitable for distributed environments.

Future work may investigate cluster-scale variants of ARS in which spatial bins are distributed across nodes and refined independently. Such architectures could potentially combine local cache-efficient classification with distributed merge-free aggregation strategies.

E. Accelerator-Aware Implementations

The ARS execution model consists primarily of histogram generation, prefix-sum computation, and spatial scattering, all of which map naturally to massively parallel hardware.

Future implementations may investigate GPU, FPGA, and heterogeneous CPU-GPU variants of the framework. In particular, the division-free classification model employed by Aero appears well suited to SIMD and SIMT execution environments where predictable control flow is essential for maintaining high throughput.

F. Persistent Streaming Architectures

Exp D demonstrates that spatial classification can support high-throughput streaming ingestion through epoch-based micro-batching and concurrent spatial consolidation. However, the current design remains an in-memory architecture.

Future research may investigate persistent spatial registries, fault-tolerant consolidation pipelines, and disk-backed run management systems. Such extensions could enable ARS to serve as the foundation for large-scale streaming databases, event-processing systems, and real-time analytical platforms.

G. Learned Spatial Mapping Functions

Aero currently utilizes a sampled quantile lookup table to approximate distribution-aware partitioning. An alternative direction involves replacing static lookup tables with lightweight learned models—drawing inspiration from learned database indexes like the Recursive Model Index (RMI) [14] and learned sorting algorithms like LearnedSort [15]—capable of predicting optimal spatial mappings directly from observed data characteristics.

The integration of learned mapping functions may further improve occupancy balance, reduce sampling overhead, and enable adaptive behavior under rapidly changing distributions while preserving the constant-time classification properties of the framework.

Overall, the exploratory architectures and extensions presented throughout this work suggest that ARS should be viewed not as a single sorting algorithm, but as a broader family of hardware-conscious spatial processing architectures. The Aero and Exp D implementations represent only a subset of this design space, leaving substantial opportunities for future investigation across parallel, distributed, and streaming data-processing environments.

VIII. Conclusion

This paper introduced Adaptive Range Sorting (ARS), a hardware-conscious spatial classification framework designed to address the growing impact of memory hierarchy behavior, branch prediction costs, and parallel scalability constraints in modern sorting systems.

Rather than approaching sorting as a comparison-driven decision process, ARS transforms the problem into one of spatial classification through monotonic key projection, histogram-based partitioning, and contention-free scatter execution. This perspective enables the framework to leverage predictable control flow, contiguous memory movement, and hardware-efficient partitioning strategies that align naturally with contemporary multi-core architectures.

The paper presented the evolutionary development of the ARS lineage, tracing its progression from recursive spatial trees to the Aero architecture. Through the introduction of a cache-resident quantile mapping table and division-free classification pipeline, Aero improves occupancy stability under skewed distributions while maintaining high-throughput parallel execution. The work also explored several experimental architectural branches that investigated recursive refinement, hierarchical memory staging, adaptive execution policies, and streaming-oriented processing.

To support continuous data ingestion, the Exp D architecture extended the framework beyond conventional batch sorting through epoch-based micro-batching and concurrent spatial consolidation. This demonstrated that the same spatial classification principles used for parallel sorting can also be applied to streaming workloads while maintaining bounded memory growth and stable ingestion throughput.

Theoretical analysis showed that recursive spatial partitioning bounds computational work to linear complexity under fixed-width key assumptions, while practical analysis highlighted the importance of memory hierarchy behavior, cache residency, and branch reduction in determining real-world performance. Experimental evaluation across diverse workloads demonstrated competitive throughput, strong scalability characteristics, robustness under skewed distributions, and favorable microarchitectural behavior relative to modern comparison-based baselines.

The results presented throughout this work suggest that spatial classification represents a viable alternative execution model for high-throughput data processing. Rather than viewing sorting exclusively through the lens of comparison-based decision trees, ARS demonstrates that hardware-conscious partitioning, adaptive classification, and spatial organization can provide an effective foundation for parallel and streaming systems operating on modern computing platforms.

More broadly, the exploratory architectures investigated throughout this study indicate that ARS should be viewed not as a single algorithm, but as a family of spatial processing architectures. The Aero and Exp D implementations represent only two points within a larger design space encompassing adaptive execution, hierarchical memory optimization, distributed processing, and accelerator-oriented execution models. As hardware continues to evolve toward increasing parallelism and memory complexity, spatial classification may provide a promising direction for the next generation of scalable data-processing systems.

REFERENCES

[1] C. A. R. Hoare, "Quicksort," *The Computer Journal*, vol. 5, no. 1, pp. 10–16, Jan. 1962.

[2] M. Frigo, C. Leiserson, H. Prokop, and S. Ramachandran, "Cache-oblivious algorithms," in *40th Annual Symposium on Foundations of Computer Science (Cat. No.99CB37039)*. New York City, NY, USA: IEEE Comput. Soc, 1999, pp. 285–297.

[3] Wm. A. Wulf and S. A. McKee, "Hitting the memory wall: Implications of the obvious," *ACM SIGARCH Computer Architecture News*, vol. 23, no. 1, pp. 20–24, Mar. 1995.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, fourth edition ed. Cambridge, Massachusett: The MIT Press, 2022.

[5] K. Kaligosi and P. Sanders, "How Branch Mispredictions Affect Quicksort," in *Algorithms – ESA 2006*, D. Hutchison, T. Kanade, J. Kittler, J. M. Kleinberg, F. Mattern, J. C. Mitchell, M. Naor, O. Nierstrasz, C. Pandu Rangan, B. Steffen, M. Sudan, D. Terzopoulos, D. Tygar, M. Y. Vardi, G. Weikum, Y. Azar, and T. Erlebach, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, vol. 4168, pp. 780–791.

[6] M. Axtmann, S. Witt, D. Ferizovic, and P. Sanders, "In-Place Parallel Super Scalar Samplesort (IPSSSSo)," *LIPIcs, Volume 87, ESA 2017*, vol. 87, pp. 9:1–9:14, 2017.

[7] G. E. Blelloch, "Prefix sums and their applications," p. 1294199 Bytes, Jun. 2018.

[8] R. D. Blumofe and C. E. Leiserson, "Scheduling multithreaded computations by work stealing," *Journal of the ACM*, vol. 46, no. 5, pp. 720–748, Sep. 1999.

[9] L. Devroye, *Non-Uniform Random Variate Generation*. New York, NY: Springer New York, 1986.

[10] G. Graefe, "Implementing sorting in database systems," *ACM Computing Surveys*, vol. 38, no. 3, p. 10, Sep. 2006.

[11] C. Nyberg, T. Barclay, Z. Cvetanovic, J. Gray, and D. Lomet, "Alphasort: A cache-sensitive parallel external sort," *The VLDB Journal*, vol. 4, no. 4, pp. 603–627, Oct. 1995.

[12] M. Zaharia, T. Das, H. Li, T. Hunter, S. Shenker, and I. Stoica, "Discretized streams: Fault-tolerant streaming computation at scale," in *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*. Farminton Pennsylvania: ACM, Nov. 2013, pp. 423–438.

[13] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The log-structured merge-tree (LSM-tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351–385, Jun. 1996.

[14] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, "The case for learned index structures," in *Proceedings of the 2018 International Conference on Management of Data*, 2018, pp. 489–504.

[15] A. Khalifa *et al.*, "Learnedsort: A non-comparison-based sorting algorithm using neural networks," *arXiv preprint arXiv:2003.12883*, 2020.